

Organigramme détaillé du point d'entrée du MOTEUR Z80LIB

Documentation complète du flux d'exécution, depuis le démarrage du programme jusqu'à la boucle de jeu à 50 Hz.

Sommaire

1. VUE D'ENSEMBLE
 2. Z8oLIB LANCER — SÉQUENCE D'INITIALISATION
 3. Étape 1 : ETEINDRE ENCRES
 4. Étape 2 : ATTENTE VBL INITIALE
 5. Étape 3 : CONFIGURER CRTC
 6. Étape 4 : DEMARRER IM₁
 7. Étape 5 : CHARGER SCENE INITIALE ID=0
 8. BOUCLE PRINCIPALE : attente *FRAMEREADY* et exécution de *SCENEACTUEL*
-

1. Vue d'ensemble

Z80LIB

À propos

Z8oLIB est un moteur de jeu écrit en assembleur Z80 pour Amstrad CPC, créé par **Patrick MAES** dans un cadre éducatif.

Il a pour vocation d'accompagner l'apprentissage de la programmation bas niveau sur CPC : gestion de l'écran en Mode 0, synchronisation via les interruptions, architecture de scènes, déplacement de sprites, et optimisation du code Z80.

Ressources pédagogiques

Des vidéos explicatives présentant le fonctionnement du moteur et les concepts abordés sont disponibles sur la chaîne YouTube dédiée :

🔗 <https://www.youtube.com/@DevZ8oCPC>

Crédits

- **Auteur** : Patrick MAES
- **Contexte** : projet éducatif
- **Chaîne YouTube** : @DevZ8oCPC

```
Initialisation (une seule fois)
    Éteindre les encres
        ↓
    Attente VBL
        ↓
    Configurer le CRTC
        ↓
    Démarrer IM1
        ↓
    Charger scène initiale
        ↓
Boucle principale (tourne à 50 Hz jusqu'à extinction)
    Attend FRAME_READY == 0
        ↓
    Exécute SCENE_ACTUEL (logique + affichage de la frame)
        ↓
    Libère FRAME_READY
        ↓
    ↺ retour à l'attente
```

Cinq étapes d'initialisation s'exécutent une seule fois, puis la boucle principale prend le relais indéfiniment, cadencée par les interruptions IM1 qui détectent le vrai retour vertical de l'écran.

2. FZ80LIBLANCER — séquence d'initialisation

```
F_Z80LIB_LANCER
    CALL F_ETEINDRE_ENCRES      ; Mettre les encres en noire
    CALL F_ATTENTE_VBL         ; Attendre synchronisation verticale
    CALL F_CRTC_INITIALISER    ; Configurer le CRTC
    CALL F_IM1_START           ; Lancer interruption IM1
    LD    B, 0
    CALL F_SCENE_CHARGER
    LD HL, FRAME_READY         ; Drapeau synchronisation 50Hz
BOUCLE_PRINCIPALE
    LD    A, (HL)              ; Charge la valeur du drapeau
    AND    A                   ; Teste si drapeau est égal à 0
    JR    NZ, BOUCLE_PRINCIPALE ; Si ce n'est pas 0, on boucle en attendant
    SCENE_ACTUEL: CALL #0000
    LD    HL, FRAME_READY
    INC    (HL)                ; On remet le drapeau à 1 (bloquant)
    JR    BOUCLE_PRINCIPALE    ; On repart pour un tour de boucle
```

C'est le point d'entrée unique du programme. Chaque étape est justifiée par un ordre précis :

Étape	Rôle	Pourquoi cet ordre
Éteindre les encres	Écran noir dès le départ	Évite un flash de couleurs parasites pendant que le reste s'initialise
Attente VBL	Synchro avant reconfig	Évite de reprogrammer le CRTC en pleine partie affichée (écran déchiré)
Configurer CRTC	Géométrie écran	Fait juste après le VBL, dans la fenêtre de non-affichage
Démarrer IM1	Active la synchro 50Hz	Une fois l'écran stable
Charger scène initiale	Prépare SCENEACTUEL / IM1CURRENT	Juste avant d'entrer dans la boucle infinie

3. Étape 1 : FETEINDREENCRES

```
F_ETEINDRE_ENCRES
    LD    A, 17                ; On commence par la 17e encre (16 = Bordure)
    LD    B, HI(GA_PORT)      ; B = port Gate Array
    LD    D, HW_BLACK         ; D = couleur hardware noir

.B_ENCRES
    DEC   A                    ; Encre précédente (gère les flags automatiquemen
t)
    mGA_SET_PEN_INK (VOID)    ; Définit encre A avec la couleur D
    JR    NZ, .B_ENCRES       ; Boucle tant que A != 0 (de 16 à 1)
    RET
```

Éteint les 17 encres du Gate Array (16 encres + bordure). `B` porte le port Gate Array pendant toute la boucle — c'est pour ça que le compteur utilise `A` / `DEC A` plutôt que `B` / `DJNZ` : `mGA_SET_PEN_INK` a besoin que `B` reste stable pour ses deux `OUT (C), x`.

Coût : 14 octets, exécutée une seule fois — aucun enjeu de performance.

4. Étape 2 : Attente VBL initiale

```
WaitVSync:
    ld    bc, #F500

.vs1:
    in    a, (c)
    rra
    jr    nc, .vs1

.vs2:
    in    a, (c)
    rra
    jr    c, .vs2
    ret
```

Attente bloquante d'un vrai retour vertical, avant de toucher au CRTC. Deux phases : attendre que le signal retombe (fin d'un VBL éventuellement déjà en cours), puis attendre le vrai flanc montant du VBL suivant — évite un faux-positif si le signal est déjà actif au moment de l'appel.

5. Étape 3 : FCRTCINITIALISER

```
F_CRTC_INITIALISER
    LD    HL, CRTC_TABLE      ; HL = pointeur vers les 14 valeurs à écrire
    LD    C, 13               ; C = 13 -> point de départ (registre R13)
    mCRTC_SCR_SETUP (VOID)    ; boucle d'écriture registre par registre
    RET
```

Configure les 14 registres du CRTC 6845 (résolution, timing, adresse écran) via la macro `mCRTC_SCR_SETUP`, qui exploite `C` à la fois comme numéro de registre et compteur de boucle, et l'astuce `OUTI` (décrémente `B` avant d'écrire) pour économiser des instructions.

Géométrie confirmée par CRTC_TABLE.INC

Constante	Valeur	Cohérence
<code>SCR_PX_WIDTH_M0</code>	128	limites balle/raquette (<code>RIGHT_LIMIT=126</code>)
<code>SCR_PX_HEIGHT_M0</code>	192	limites verticales (<code>BOTTOM_LIMIT=188</code> , <code>PADDLE_PX_BOTTOM_LIMIT=168</code>)
<code>SCR_WIDTH_BYTE</code>	64 (<code>#40</code>)	stride utilisé dans tous les calculs d'adresse VRAM
<code>SCR_VRAM_BASE</code>	<code>#C000</code>	base utilisée dans toutes les macros de calcul d'adresse

Coût : 19 octets (3+2+13+1).

6. Étape 4 : FIM1START et le vecteur IM1_CALLBACK

Installation du vecteur

```
F_IM1_START
    DI                                     ; Désactive le temps de la modif (non-atomique)
    LD    HL, F_IM1_CALLBACK
    LD    (#0039), HL
    EI                                     ; Réactive globalement les interruptions
    RET
```

Redirige le `JP nnnn` déjà présent en `#0038` (placé par le firmware) vers `F_IM1_CALLBACK`, en écrivant uniquement son opérande. Le `DI` protège cette écriture non-atomique (2 octets) contre une interruption qui surviendrait pile au milieu.

La routine d'interruption elle-même

```
F_IM1_CALLBACK
    EXX
    EX    AF, AF'
    PUSH  IX
    PUSH  IY

    LD     A, HI(PPI_PORT_B)
    IN     A, (0)
    RRCA
    JR     NC, .NO_VBL

    LD     HL, FRAME_READY
    XOR    A
    LD     (HL), A
    JR     .DISPATCH

.NO_VBL
    LD     A, (IM1_INDEX)
    CP     IM1_NOT_READY
    JR     Z, F_IM1_SKIP

    INC    A
    CP     IM1_IT_MAX
    JR     C, .DISPATCH
    XOR    A

.DISPATCH
    LD     (IM1_INDEX), A
    ADD    A, A
    LD     C, A
    LD     B, 0
    LD     HL, (IM1_CURRENT)
    ADD    HL, BC
    LD     A, (HL)
    INC    HL
    LD     H, (HL)
    LD     L, A
    JP     (HL)

; -----
; DONNÉES POUR LE SYSTÈME D'INTERRUPTIONS VECTORISÉES
```

```

; -----
IM1_INDEX:
    DEFB 0                                ; Valeur initiale = -2 ou 0xFE
                                           ; Indique que l'index est "non prêt"
                                           ; Pas encore d'interruption à traiter

IM1_CURRENT:
    DEFW 0                                ; Pointe vers le tableau de vecteurs par défaut

```

Déclenchée ~300 fois/seconde (cadence CRTC), cette routine :

1. Sauvegarde tout le contexte (registres principaux et alternatifs).
2. Relit le port PPI pour détecter un **vrai** VBL, corrigeant toute dérive du timing CRTC.
3. Si VBL : libère `FRAME_READY`.
4. Sinon : avance `IM1_INDEX` et dispatche vers la sous-tâche IM1 courante, via la table `IM1_CURRENT` (propre à chaque scène, mise à jour par `F_SCENE_CHARGER`).

Principe important : cette routine doit rester minimale. Tout traitement additionnel (compteur de frame, timing de jeu) doit être déplacé vers la scène elle-même, exécutée à 50 Hz plutôt que dans l'ISR à ~300 Hz.

7. Étape 5 : FSCENECHARGER (scène initiale ID=0)

```
F_SCENE_CHARGER
    LD    A, IM1_NOT_READY
    LD    (IM1_INDEX), A
    LD    A, B
    CP    SCENE_TABLE_COUNT
    RET   NC

    ADD   A, A
    LD    E, A
    ADD   A, A
    ADD   A, E
    LD    L, A
    LD    H, 0
    LD    DE, SCENE_TABLE
    ADD   HL, DE

.scene_found
    LD    E, (HL)
    INC   HL
    LD    D, (HL)
    INC   HL
    PUSH  DE
    LD    E, (HL)
    INC   HL
    LD    D, (HL)
    INC   HL
    LD    (IM1_CURRENT), DE
    LD    E, (HL)
    INC   HL
    LD    D, (HL)
    POP   HL
    LD    (SCENE_ACTUEL+1), HL
    PUSH  DE
    RET
```

Avec `B=0`, le calcul d'offset (`ID*6`) donne `0` — `HL` pointe directement sur la première entrée de `SCENE_TABLE` (table à 6 octets/entrée : 3 pointeurs de 2 octets chacun).

Ce qui est mis à jour

Variable	Valeur après ID=0
IM1_INDEX	IM1_NOT_READY (jusqu'au prochain vrai VBL)
IM1_CURRENT	adresse de la table IM1 propre à la scène 0
SCENE_ACTUEL+1	adresse de la routine logique de la scène 0

Le saut final se fait via `PUSH DE : RET` (équivalent d'un `JP (DE)`, absent nativement sur Z80) vers la routine d'initialisation de la scène.

8. Boucle principale : attente FRAMEREADY et exécution de SCENEACTUEL

```
BOUCLE_PRINCIPALE
    LD    A, (HL)
    AND  A
    JR    NZ, BOUCLE_PRINCIPALE
    SCENE_ACTUEL: CALL #0000
    LD    HL, FRAME_READY
    INC  (HL)
    JR    BOUCLE_PRINCIPALE
```

Attente active

Boucle serrée qui relit `FRAME_READY` jusqu'à ce qu'il tombe à `0` (mis par `F_IM1_CALLBACK` lors d'un vrai VBL). Le CPU reste disponible pour l'interruption IM1 pendant cette attente.

Exécution via code auto-modifiant

`SCENE_ACTUEL: CALL #0000` — l'opérande de ce `CALL` est réécrit dynamiquement par `F_SCENE_CHARGER`. La boucle reste ainsi totalement générique, indépendante du nombre de scènes du jeu.

Libération du drapeau

`INC (HL)` remet `FRAME_READY` à une valeur non nulle, relançant l'attente pour la frame suivante.

9. Ce qui s'exécute dans SCENE_ACTUEL (scène de jeu)

Pour la scène de jeu, `SCENE_ACTUEL` enchaîne dans l'ordre :

- 1. ****Initialisation** : Paramétrage de la scène sera lancer une seule fois.
 - 2. **Logique** : Calculer les données (positions, scores, états, ...).
- Affichage** : Chaque entité, effacement à l'ancienne position VRAM puis redessin à la nouvelle.

10. Points de vigilance identifiés

Risque	Description	Mitigation
Dépassement du budget de frame	Si <code>SCENE_ACTUEL</code> dépasse ~20ms, un VBL peut survenir en plein traitement, masqué silencieusement par <code>INC (HL)</code>	Optimisations : <code>VramAddr</code> incrémentale, <code>LDI</code> , suppression des tables inutiles
ISR alourdie	Ajouter un compteur de frame ou du timing dans <code>F_IM1_CALLBACK</code> ralentirait les ~300 appels/seconde	Déplacer ce genre de logique dans la scène elle-même, exécutée à 50 Hz
Écriture non-atomique du vecteur IM1	<code>LD (#0039),HL</code> peut être interrompue en plein milieu	<code>DI</code> / <code>EI</code> autour de l'écriture dans <code>F_IM1_START</code>